



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



SERLAB
Software Engineering Research

Ingegneria del Software

Principi dell'Ingegneria del Software

ITPS A.A. 2025/2026

Vita Santa Barletta
Danilo Caivano
Antonio Piccinno

Dipartimento di Informatica - Università degli Studi di Bari
Via Orabona, 4 - 70125 - Bari
Tel: +39.080.5443270 | Fax: +39.080.5442536
serlab.di.uniba.it



PRINCIPI dell'Ingegneria del Software



Principi dell'ingegneria del software

- I **principi** formano le **basi** dei **metodi, tecniche, metodologie** e **strumenti**
- **Sette principi** sono **fondamentali** in tutte le fasi dello sviluppo software:
 - ☐ Produzione
 - ☐ Manutenzione
- La **modularità** è la pietra angolare della **progettazione** dei sistemi software

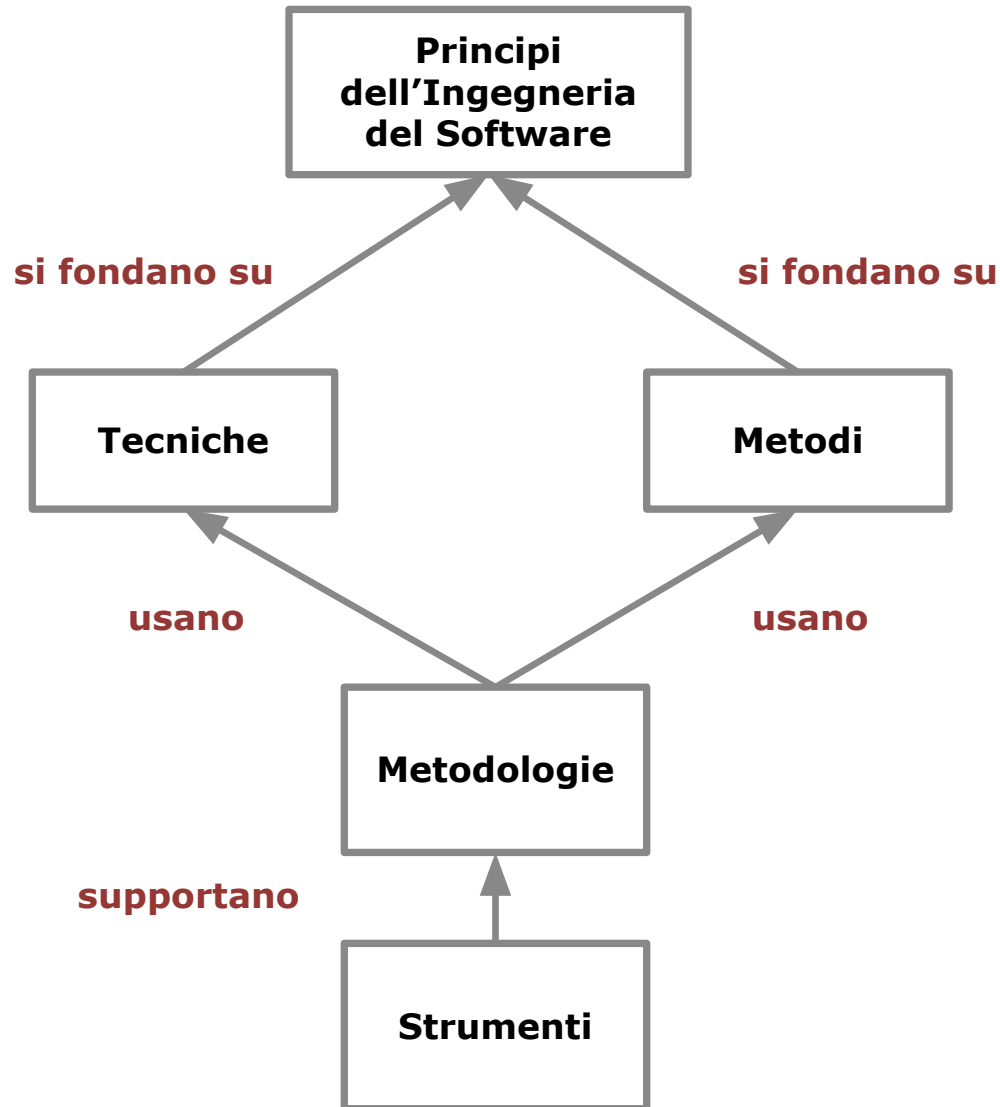


Applicazione dei principi

- I principi descrivono proprietà desiderabili dei **prodotti** e dei processi in termini **generali** ed **astratti**
- I principi sono **applicabili in tutti i processi di sviluppo** del software, indipendentemente da:
 - ☐ Il contesto di sviluppo
 - ☐ Il tipo di sistema da sviluppare
 - ☐ L'ambiente di sviluppo
 - ☐ L'ambiente target del sistema software
- I principi si trasformano in **pratiche** attraverso **metodi** e **tecniche**
 - ☐ Spesso i metodi e le tecniche sono confezionate in **metodologie**
 - ☐ Le metodologie possono essere supportate dagli **strumenti** che **potenziano gli sviluppatori**



Rappresentazione grafica delle relazioni



RIGORE E FORMALITÀ

1



Concetti

- L'**Ingegneria del Software** è un'**attività creativa**, ma deve essere praticata **sistematicamente**
- Il **RIGORE** è il giusto complemento alla creatività per aumentare la **confidenza in tutto ciò che si sviluppa**
- Il **Rigore** è la coerenza con le pre-condizioni e con lo scopo del manufatto che si sta producendo oltre che con il metodo e le tecniche che si intendono utilizzare nella produzione
 - ❑ Inflessibilità, esattezza, precisione assoluta
- Esistono **diversi livelli di rigore**, quello più alto è la **FORMALITÀ**
- **Formalità** è l'uso esclusivo di formalismi matematici per guidare lo sviluppo del software e per valutarlo



Esempio di prodotto

- Precondizione
 - ❑ $\exists$ un **Volo** con un **tempo di Durata** prevista
 - ❑ Il volo è classificabile come: **Nazionale, Intracontinentale, Intercontinentale**
- Postcondizione
 - ❑ Definiti i Servizi di Ristoro da erogare
- **Rigoroso**
 - ❑ Se Durata del Volo < 1hh: distribuire bevande
 - ❑ Se Durata del Volo >1 hh e Tipo di Volo = Intracontinentale: Distribuire Bevande e Snack
 - ❑ Se Durata del Volo >1hh e Tipo di Volo = Intercontinentale: Distribuire Bevande e Pranzo
 - ❑ Se Durata Volo > 8hh e Tipo di Volo = Intercontinentale: Distribuire Bevande, Pranzo e Colazione.
- Consente di **derivare rigorosamente e sistematicamente i dati per la verifica dinamica**



Esempio di prodotto

- **Formale**

- **Condizioni**

- Durata Volo : [1,8 [
- Tipo di Volo :[Nazionale; Intracontinentale; Intercontinentale]

- **Azioni**

- Distribuzione :[Bevande; Snack; Pranzo; Colazione]
- Distribuzione = F (Durata Volo, Tipo Volo)

1. Durata Volo	< 1		[1 ... 8]			> 8	
2. Tipo Volo	Nazionale	Intracontinentale or Intercontinentale	Nazionale	Intracontinentale	Intercontinentale	Nazionale or Intracontinentale	Intercontinentale
1. Bevanda	x	.	.	x	x	.	x
2. Snack	.	.	.	x	.	.	.
3. Pranzo	.	.	.	.	x	.	x
4. Colazione	.	.	.	.	.	.	x
	1	2	3	4	5	6	7



Esempio di processo

- La **descrizione rigorosa delle attività** e delle **relazioni tra loro** da eseguire durante lo sviluppo del software aiuta a:
 - ❑ **gestire il progetto**
 - ❑ **accertare i tempi** ed i **costi del progetto**



Vantaggi

- Il **rigore consente di sviluppare software con buoni livelli di qualità**
 - La scrittura rigorosa della documentazione la rende comprensibile e tracciabile, quindi l'applicazione software ha una buona manutenibilità
- Il **formalismo consente la gestione automatica dello sviluppo software**
 - Un programma specificato formalmente può essere generato automaticamente
 - Un processo descritto formalmente può essere **monitorato** automaticamente e se non sono implicate attività umane può essere **eseguito automaticamente**
- I concetti trattati nello sviluppo di un'applicazione sono ad **alto contenuto semantico**, quindi sono difficilmente formalizzabili
- Pertanto è necessario che siano **almeno rigorosi** per evitare che si realizzi software con dissonanze concettuali; questo è un rischio che quando si verifica **genera molte diseconomie**

Svantaggi

- **È difficile** per uno sviluppatore **mantenere** sistematicamente lo stesso livello di **rigore**; spesso egli si comporta **non conformemente**
- Il **formalismo richiede** molto **impegno persona**



SEPARAZIONE DEGLI INTERESSI

2



Concetti

- **Separare le decisioni** necessarie per risolvere **aspetti diversi** dello stesso problema
- Esistono diverse modalità di **separazione**:
 - ❑ **Per Tempo**. Aspetti che si rilevano in tempi diversi
 - ❑ **Per Caratteristiche**. Ogni aspetto rilevante corrisponde ad una caratteristica, ognuna è analizzata separatamente dall'altra
 - ❑ **Per Prospettive**. Ogni aspetto attiene alla prospettiva di un portatore di interessi
 - ❑ **Per Parti**. Ogni capacità del sistema deve riguardare una parte di esso



Esempi di prodotto e di processo

- **Separazione per Tempo**. I problemi e le attività dell'analisi si separano da quelli della progettazione
- **Separazione per Caratteristiche**. Si progetta prima il sistema software con una elevata **qualità della struttura**. Successivamente, se fosse necessario, si affina la struttura perché sia **efficiente** quanto è richiesto
- **Separazione per Prospettive**. Si struttura il sistema per assicurare una buona **usabilità**; si rivede poi la struttura per assicurare una **manutenzione economica**; si implementano le specifiche riusando software disponibile per assicurare **economia nello sviluppo**; si progetta l'interfaccia utente per rendere **usabile** il sistema; si prova il sistema per verificare quanto sia **efficiente**
- **Separazione per Parti**. Le applicazioni devono essere progettate con le seguenti parti:
 - ❑ **Gestire l'interfaccia**
 - ❑ **Gestire le regole di dominio** applicativo
 - ❑ **Gestire i dati persistenti**
 - Ognuna di queste parti deve essere suddivisa, sino ad arrivare alle **capacità prime** che costituiranno l'architettura



Vantaggi

- I problemi da affrontare nello sviluppo di un'applicazione software sono di diversi tipi ed in diversi domini. Tutti richiedono compromessi specifici
- L'applicazione di questo principio consente di **analizzare, uno per volta, i differenti aspetti/interessi** di un grande problema, **diminuendo così il numero di decisioni da assumere contemporaneamente**

Svantaggi

- Separare gli aspetti potrebbe far **perdere la ottimizzazione** perché si assumono **decisioni** riguardanti aspetti differenti che potrebbero trascurare la **relazione esistente tra alcune di esse**
 - Per questo motivo **due o più aspetti con caratteristiche comuni conviene non separarli**

MODULARITÀ

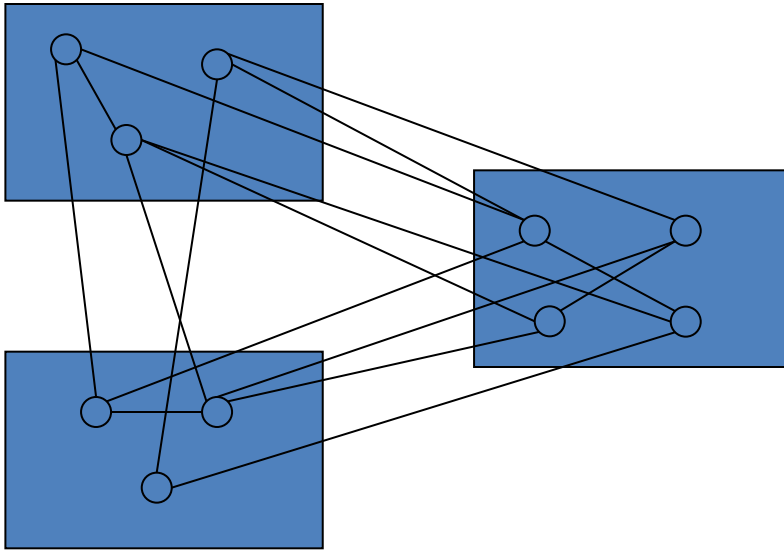
3

Concetti

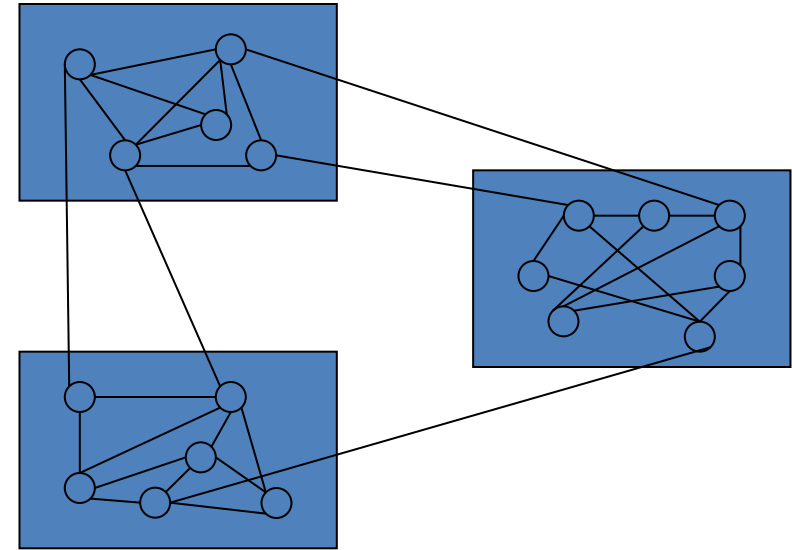
- I Moduli sono **parti del Sistema Software**
 - ❑ Ognuno realizza una parte delle sue funzioni
 - ❑ Tutti **cooperano** tra loro per realizzare gli **scopi del Sistema Software**
- I moduli devono avere
 - ❑ **Alta coesione interna**: misura la forza che giustifica la coesistenza logica degli elementi interni di un modulo (istruzioni, procedure, metodi...)
 - ❑ **Basso accoppiamento tra loro**: misura la densità ed il tipo di interdipendenza tra moduli
- **Un'applicazione si può costruire secondo due vie**
 - ❑ Si scompone il problema in moduli e successivamente si dettaglia il progetto di ognuno di essi: approccio **Top Down**
 - ❑ Si parte dai moduli in dettaglio e si compongono per ricavare moduli sempre più grandi: approccio **Bottom Up**



Esempi



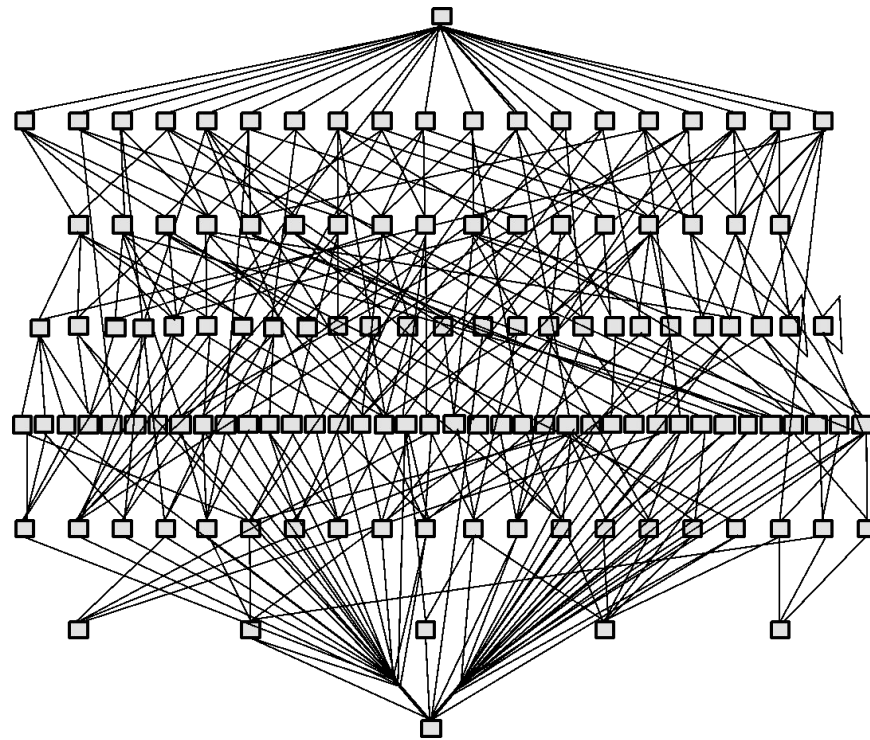
Struttura ad
alto accoppiamento e
bassa coesione interna



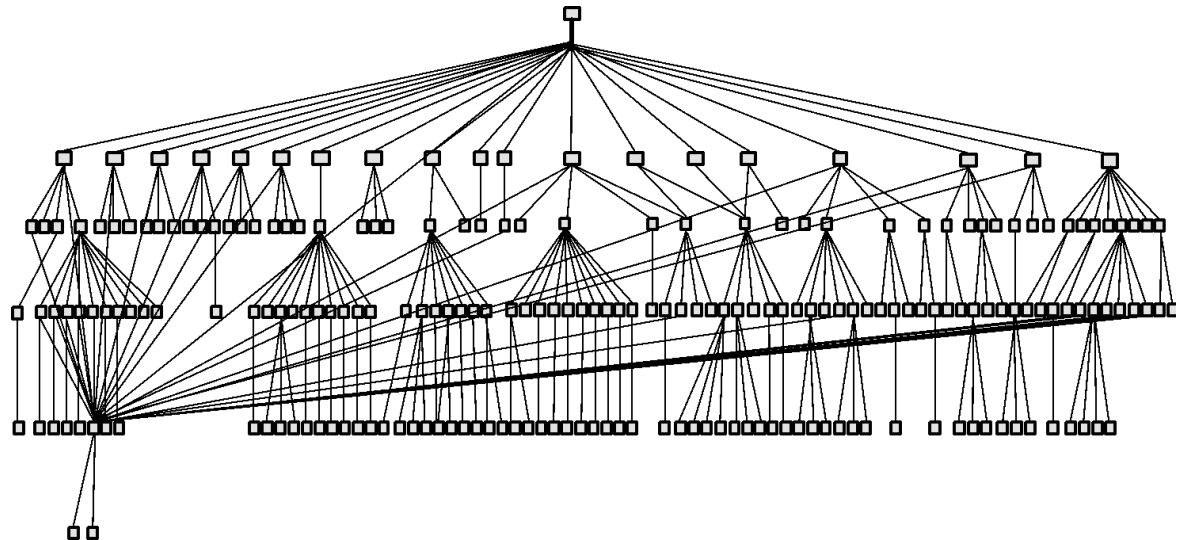
Struttura con
alta coesione e
basso accoppiamento

Esempi

Programma ad alto
accoppiamento



Stesso programma
con l'accoppiamento
migliorato



Vantaggi

- I Moduli consentono di applicare la **separazione degli interessi**:
 - ❑ Prima sono trattati gli **scopi** di ogni modulo e le relazioni tra loro in modo dettagliato
 - ❑ Dopo si analizzano i **dettagli** di ogni modulo
 - ❑ Infine si verifica la **coerenza** della loro integrazione attraverso le relazioni e la correttezza del sistema integrato rispetto agli scopi complessivi del sistema
- I Moduli possono essere componenti già scritte che sono catalogate in una libreria e che sono **riusate** nella costruzione del Sistema Software consentendo forti economie
- I moduli consentono di **comprendere il sistema** per servizi resi da un modulo e per relazioni tra i servizi
 - ❑ Questo agevola la **manutenzione**



Svantaggi

- La eccessiva divisione in moduli può **diminuire la comprensibilità** del Sistema Software
- In modo particolare quando le funzioni sono state divise tra molti moduli per rispettare la Separazione degli Interessi
 - ❑ Esempio:
 - Una funzione che ha bisogno di accedere a molti dati potrebbe essere parcellizzata tra i moduli che realizzano l'accesso ad una parte di dati e la relativa elaborazione
 - I risultati di questi moduli sono dati semilavorati in ingresso ad un altro modulo che da essi ricava il risultato della funzione
 - In questo caso **la funzione è stata divisa tra molti moduli** e risulterà **difficile la sua comprensione**

4

ANTICIPAZIONE DEL CAMBIAMENTO



Concetti

- Capacità di un'applicazione di **accogliere i cambiamenti** generati da qualsivoglia causa, con poco impegno, in poco tempo e con un limitato rischio
- I cambiamenti sono generati da:
 - ❑ Miglioramento della **comprensione del Dominio Applicativo**
 - ❑ Miglioramento della **comprensione dei modelli utilizzati nello sviluppo del software**
 - ❑ **Cambiamenti** del **Dominio Applicativo**
 - ❑ **Cambiamenti** delle **tecnologie** del Dominio delle Soluzioni
- Questo principio è quello che **differenzia** considerevolmente **lo sviluppo del software** con la **produzione di tutti gli altri tipi di beni**

Esempio

- Un programma per la gestione di un catalogo che:
 - ❑ Deve comprendere quando i dispositivi di memorizzazione in linea hanno raggiunto un **livello di** riempimento, di **"guardia"**
 - ❑ Deve **salvare** un set di records dai dispositivi di **memorie in linea** a dispositivi di **memorie fuori linea**
 - ❑ **Recuperare dai dispositivi fuori linea** alcuni record richiesti che non sono più in linea
- È opportuno che:
 - ❑ Abbia il **livello di guardia definito da differenti algoritmi**; gli algoritmi tengano conto delle capacità disponibili in ogni impianto utilizzatore
 - ❑ Abbia **diverse modalità per selezionare i record da salvare** dalle memorie on line a quelle off line; abbia la possibilità di produrre un numero di copie di record salvati, opportunamente identificati; abbia la possibilità di **spostare** records **da memorie off line più pregiate** ad altre meno pregiate, nel caso che l'utente ne abbia bisogno
 - ❑ Abbia **algoritmi differenti per attivare il recupero**; inoltre abbia algoritmi differenti per individuare il set di record da recuperare e modalità differenti di trattare i record recuperati, dopo che saranno utilizzati
 - ❑ Per tutti gli **algoritmi** necessari si preveda un **set nativi** e sia possibile inserire altri algoritmi come **plug-in**



Vantaggi

- **Riduce** i tempi ed i costi per la **manutenzione**
- Aumenta la **riusabilità** delle componenti

Svantaggi

- L'**analisi e la progettazione** deve essere tanto **accurata** da prevedere, quanto più è possibile, i cambiamenti futuri
- Gli analisti devono avere **approfondita conoscenza del Dominio Applicativo** per poter prevedere i cambiamenti futuri in esso e nel mercato target dell'applicazione
- Il progetto deve avere la capacità di **accogliere i cambiamenti futuri** previsti e deve **curare l'accoppiamento dei moduli**
 - ❑ Questo deve essere tanto più piccolo quanto più alto è il livello di predisposizione al cambiamento
- Il management dell'applicazione deve essere attrezzata con processi e tools della **Gestione della Configurazione Software**

ASTRAZIONE

5

Concetti

- Estrarre ed analizzare gli **aspetti più significativi** di un problema **trascurando i dettagli** dello stesso
- I **dettagli** che si possono trascurare **dipendono dallo scopo** dell'astrazione
- **L'astrazione produce modelli** che esprimono una realtà anche complessa eliminando i dettagli che non servono alla comprensione della stessa

Esempi

- Le **applicazioni software** sono astrazioni delle **procedure manuali** che automatizzano
 - ❑ Esempio: In un programma che automatizza la fatturazione sono trascurati i particolari su come è organizzato l'ufficio fatturazione, se ha due o più persone, se è disposto su piani diversi dell'impresa, se è distribuito geograficamente
 - ❑ Realizza l'essenza del comportamento della procedura manuale, ma non ne riproduce esattamente i dettagli
- I **modelli di previsione** per un processo prendono in considerazione processi simili già eseguiti; nel confronto **si considerano solo alcuni fattori centrali** e **si trascurano i dettagli** di quanto è realmente accaduto durante la loro esecuzione



Vantaggi

- L'astrazione consente di **presentare prospettive diverse** di uno stesso sistema, **dependentemente dagli scopi** della stessa realtà
 - Esempi:
 - L'**interfaccia utente** mostra all'utilizzatore un'astrazione dell'applicazione costituita solo dai servizi che essa offre, senza dare alcuna informazione sulle componenti Hw e Sw
 - L'**analisi** esprime le capacità e le funzioni del sistema senza dare i dettagli dell'architettura, dei moduli
- L'astrazione consente di **tracciare manufatti che descrivono lo stesso sistema con diversi livelli di astrazione**, ognuno comprensibile ad una classe di parti interessate coinvolte nello sviluppo o nell'uso dell'applicazione
 - Esempio:
 - Quando sono analizzati **i requisiti di un sistema software** si produce un **modello dell'applicazione** proposta
 - È possibile **ragionare sull'applicazione** ragionando sul **modello (astratto) prodotto**



Svantaggi

- **Un'astrazione mal progettata** rispetto allo scopo potrebbe dare una **immagine deformata** dei contenuti di dettaglio
 - ❑ Esempi:
 - **Un'interfaccia poco amichevole** potrebbe far avvertire una bassa qualità della struttura di un sistema software
 - Un modulo per l'accesso ai dati persistenti che rende **complessa la navigazione nel database** fa avvertire un database progettato o strutturato male

6

GENERALITÀ



Concetti

- Durante la **risoluzione di un problema**, si prova a scoprire se esso **è una istanza di un problema più generale** la cui soluzione può essere riusata in altri casi
- Generalizzare consiste nel **reformulare un problema generale** che comprenda uno o più problemi analoghi che si devono risolvere
- Qualche volta un **problema più generale potrebbe essere più facile da risolvere** di un suo caso speciale



Esempi

- Stima dei consumi di uno strumento dinamico sia **$\text{Stima} = 4,5 * E ** (3,5 * \text{ESP})$**
- Le costanti sono state calcolate con dati derivati dall'uso di uno stesso motore per lavori analoghi
- Questo problema si può riformulare come **$\text{Stima} = a * E ** (b * \text{ESP}) + k$** . Dove:
 - ❑ (a, b, k) sono costanti che descrivono il contesto in cui è fatta la stima
 - ❑ E, ESP sono le osservabili del sistema da stimare



Vantaggi

- Consente di **riusare i moduli** che risolvono il problema generale in differenti ambienti senza modificare il codice
- Consente di trovare **prodotti di uso generale** (*off the shelf*) che possono essere inseriti nel sistema che si sta costruendo

Svantaggi

- **Maggiori costi di sviluppo** della soluzione generale

INCREMENTALITÀ

7



Concetti

- **Sviluppo graduale** di un'applicazione
 - ❑ Gli scopi finali della stessa si raggiungono per incrementi successivi
- Si può procedere **incrementalmente a tutti i livelli di astrazione**:
 - ❑ È possibile eseguire l'**analisi** di un sistema **per incrementi** e seguire con la progettazione e codifica con l'intero incremento
 - ❑ È possibile dividere il sistema analizzato in parti e per ognuna di esse fare **progettazione e codifica per incrementi**; successivamente, integrare tutti gli incrementi realizzati
 - ❑ È possibile partire da un sistema interamente progettato e **realizzarlo per incrementi**; dopo la realizzazione di tutti i moduli questi sono integrati

Esempi

- Si devono realizzare un sistema **S** con un set di funzioni **$\{F_1, F_2, F_3, \dots F_n\}$**
- Il suo sviluppo si divide in tre incrementi
 - ❑ $INCR_1 = \{F_1, F_2, F_3, \dots F_{k1}\};$
 $S_1 = INCR_1$
 - ❑ $INCR_2 = \{F_{k1}, F_{k1+1}, F_{k1+2}, \dots F_{k2}\};$
 $S_2 = \{S_1 \cup INCR_2\}$
 - ❑ $INCR_3 = \{F_{k2}, F_{k2+1}, F_{k2+2}, \dots F_n\};$
 $S = \{S_2 \cup INCR_3\}$



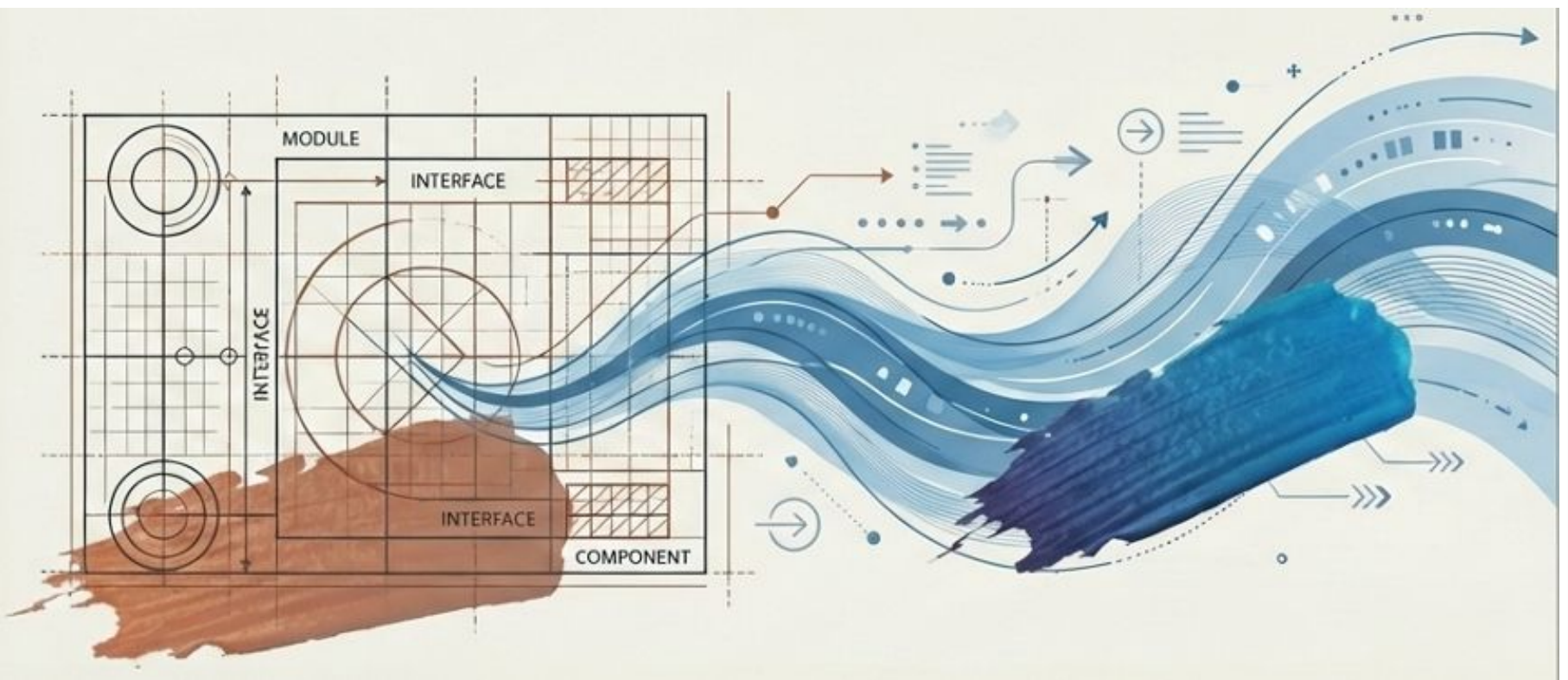
Vantaggi

- Consente di **soddisfare più rapidamente le esigenze più critiche ed urgenti** dell'utente
- Consente di **avere più frequentemente la validazione del sistema** da tutte le parti interessate
- **Diminuisce il rischio** di sviluppare un'**applicazione non rispondente ai requisiti**

Svantaggi

- **Aumentano i rischi di integrazione:** nell'integrazione di un incremento al sottosistema già sviluppato potrebbe evidenziarsi qualche relazione tra moduli delle due parti che non erano state evidenziate dal progetto
- **Aumenta il costo di test:** dopo ogni incremento è necessario fare il Test di Sistema e di Accettazione del nuovo sottosistema realizzato

Dai PRINCIPI allo STREAM CODING

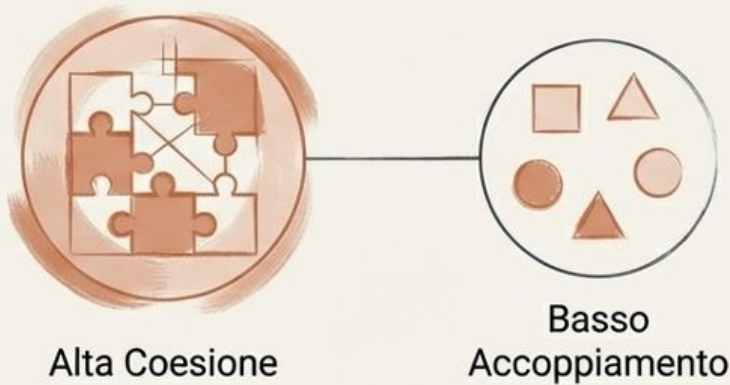


I 7 Principi Fondamentali della Progettazione



Lo Scudo contro l'Obsolescenza

Coesione vs Accoppiamento



I moduli richiedono alta coesione interna e basso accoppiamento esterno, garantiti dall'Information Hiding

Anticipazione del Cambiamento

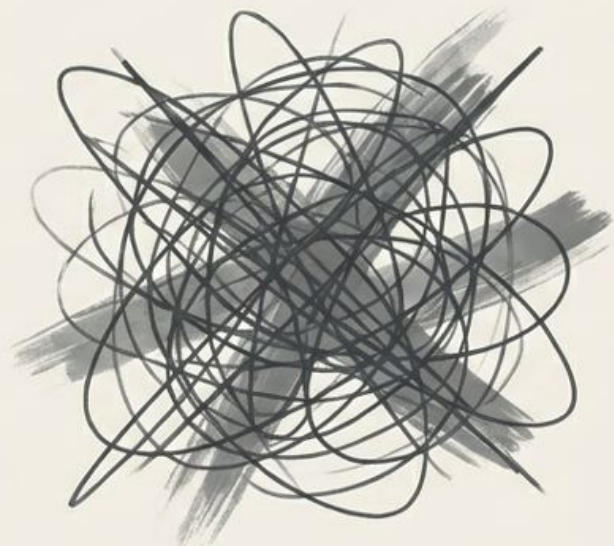


La capacità di accogliere i cambiamenti futuri con poco impegno, in poco tempo e con rischio limitato



Il Miraggio della Velocità

L'AI genera codice rapidamente, ma l'approccio conversazionale caotico sacrifica il Rigore e la Formalità (Vibe Coding)



Context Amnesia: L'AI perde il filo delle istruzioni iniziali.



Instruction Drift: Il codice devia silenziosamente dai requisiti architetturici.



Eldritch Code Horror: Codice funzionante ma totalmente incomprensibile e non manutenibile.

**La velocità non è
l'obiettivo, è un
sintomo.**

**La chiarezza è la
causa.**

La Specifica come “Source of Truth”

L'Approccio Fallimentare

Documentazione frammentata



L'AI formula assunzioni



Cicli infiniti di revisione del codice

StreamCoding

Documentazione senza ambiguità
(Documentation-First)



L'AI ha un contesto perfetto

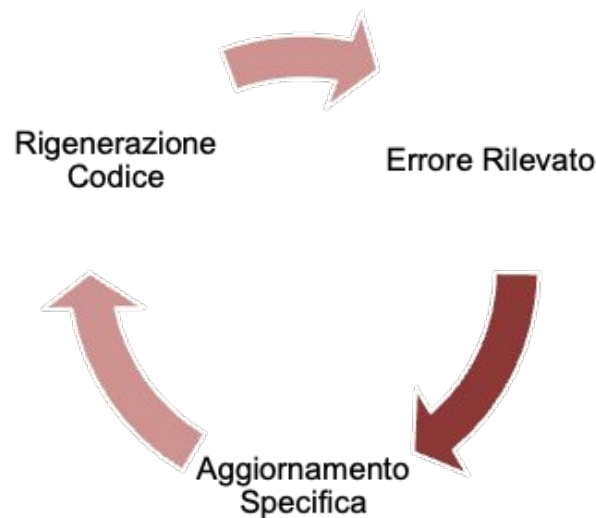


Flusso deterministico
(Il codice si scrive da solo)

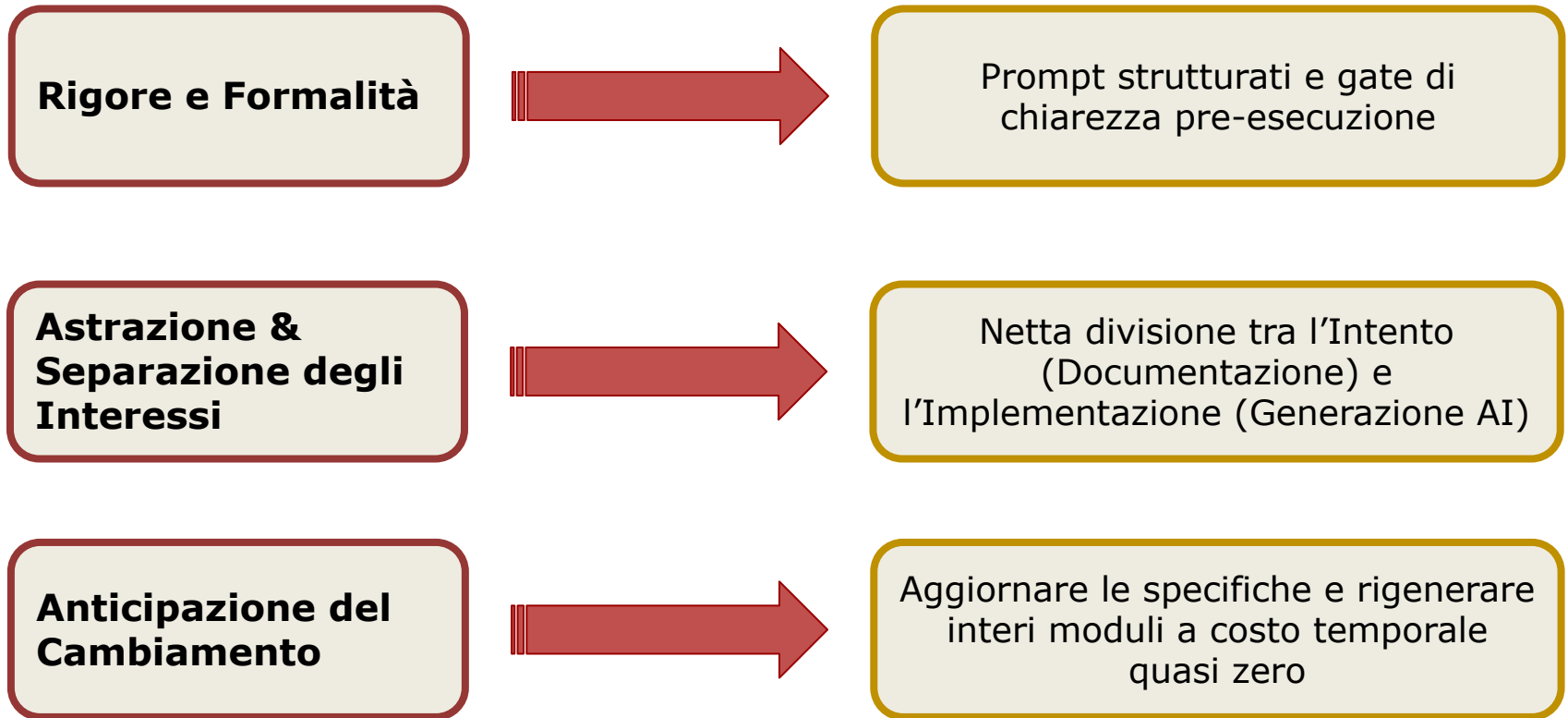
The Golden Rule: il ciclo di vita della Specifica

Se trovi un bug, correggi la SPECIFICA e rigenera

- **Il Debito di Divergenza:** una modifica manuale al codice crea divergenza
- La divergenza distrugge la sincronia tra Specifica e Implementazione, rompendo il flusso (Stream)



Lo Stream Coding e i Principi



Riferimenti

- Carlo Ghezzi, Medhi Jazayeri, Dino Mandrioli “Ingegneria del Software - Fondamenti e Principi, 2a edizione” Pearson Prentice Hall, 2004.
 - Capitolo 3: Principi dell’Ingegneria del software
- Stream Coding:
<https://github.com/frmoretto/stream-coding>

